

Xylo AI Studios

# RIA Client Onboarding Intelligence Agent

Technical Specification — Wealth Advisory (RIA) Vertical

| Field         | Value                    |
|---------------|--------------------------|
| Document Type | Full Build Specification |
| Version       | 1.0                      |
| Date          | June 2025                |
| Author        | Prasmit Pokhrel          |
| Status        | Ready for Build          |
| Deploy Target | Vercel (Free Tier)       |

## 1. Executive Summary

The RIA Client Onboarding Intelligence Agent is a multi-step, guided web application that allows boutique Registered Investment Advisor (RIA) firms to onboard new clients efficiently and consistently. An advisor walks through a smart intake form that progressively collects client data across five structured steps, then submits the data to a Claude-powered AI backend that generates a complete, professional onboarding packet in seconds.

The tool eliminates 2–4 hours of manual document preparation per client and ensures every new engagement begins with a standardised, compliance-ready deliverable set.

### 1.1 Output Deliverables

- Client Risk Profile Summary — narrative describing personality, risk tolerance, and investment experience
- Financial Goals & Constraints Brief — goals ranked by priority with timelines and constraints
- Proposed First-Year Planning Agenda — 4–6 recommended planning conversations with quarterly assignments
- Draft Investment Policy Statement (IPS) — a real compliance document the firm can review, edit, and file

## 2. Technology Stack

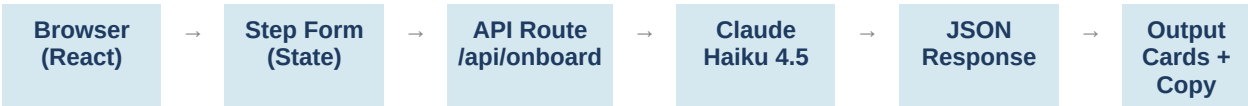
The stack is chosen for zero-to-deployed speed on a free tier with no separate infrastructure to manage.

| Layer       | Technology             | Version   | Rationale                              |
|-------------|------------------------|-----------|----------------------------------------|
| Frontend    | Next.js (App Router)   | 14+       | Fast scaffold, SSR, free Vercel deploy |
| Styling     | Tailwind CSS           | 3+        | Utility-first, no config, clean output |
| Backend     | Next.js API Routes     | Built-in  | No separate server required            |
| AI Provider | Claude API (Haiku 4.5) | Latest    | Fast, cheap, structured JSON output    |
| PDF Export  | @react-pdf/renderer    | Latest    | Client-side PDF, no server needed      |
| Deployment  | Vercel                 | Free Tier | One-command deploy, env var support    |
| Language    | TypeScript             | 5+        | End-to-end type safety                 |

## 3. System Architecture

### 3.1 Request Flow

The application follows a simple client → API route → Claude → client response pattern with no persistence layer:



### 3.2 File Structure

The project follows the Next.js 14 App Router convention:

```
ria-onboarding-agent/
├── app/
│   ├── page.tsx           # Root page – multi-step form controller
│   ├── layout.tsx         # Root layout
│   └── api/onboard/route.ts # POST endpoint – calls Claude API
├── components/
│   ├── steps/
│   │   ├── Step1BasicInfo.tsx # Name, age, marital status, employment
│   │   ├── Step2Financial.tsx  # Income, assets, accounts, liabilities
│   │   ├── Step3Goals.tsx      # Goals interview with priority ordering
│   │   └── Step4Risk.tsx       # 5-question risk assessment
│   └── ProgressBar.tsx        # Step indicator
├── lib/
│   ├── types.ts            # TypeScript interfaces
│   ├── prompt.ts           # System prompt + prompt builder
│   └── riskScorer.ts        # Maps raw answers to risk label
└── .env.local               # ANTHROPIC_API_KEY (never committed)
```

### 4. User Flow

The application progresses through five numbered steps controlled by a top progress bar. All state is held in a single React useState object in page.tsx.

| Step | Screen             | Data Collected                                                            | Navigation          |
|------|--------------------|---------------------------------------------------------------------------|---------------------|
| 1    | Client Info        | Name, age, marital status, dependents, employment, occupation             | Continue →          |
| 2    | Financial Snapshot | Income range, assets, existing accounts (multi-select), liabilities       | ← Back / Continue → |
| 3    | Goals Interview    | Up to N goals each with description, timeline, and auto-assigned priority | ← Back / Continue → |
| 4    | Risk Assessment    | 5 scenario-based questions scored 1–5 each                                | ← Back / Generate → |
| 5    | Output             | Loading spinner → four output sections with per-section Copy buttons      | Start New Client    |

## 5. Data Model

---

### 5.1 ClientFormData Interface

All form state is typed via a single ClientFormData interface defined in lib/types.ts:

```
interface ClientFormData {  
  // Step 1  
  firstName: string; lastName: string; age: number;  
  maritalStatus: 'single' | 'married' | 'divorced' | 'widowed';  
  dependents: number;  
  employmentStatus: 'employed' | 'self-employed' | 'retired' | 'other';  
  occupation: string;  
  // Step 2  
  annualIncomeRange: string; approximateAssets: string;  
  existingAccounts: string[]; liabilities: string;  
  // Step 3  
  goals: { goal: string; timeline: string; priority: number }[];  
  // Step 4  
  riskAnswers: number[]; // 5 values, each 1-5  
}
```

### 5.2 OnboardingPacket Interface

The API route returns a typed OnboardingPacket object parsed from Claude's JSON response:

```
interface OnboardingPacket {  
  riskProfile: string; // 2-3 paragraph narrative  
  goalsBrief: string; // structured goals summary  
  planningAgenda: string; // 4-6 Q1-Q4 planning items  
  draftIPS: string; // full Investment Policy Statement  
}
```

## 6. Risk Scoring Algorithm

lib/riskScorer.ts maps the five raw risk-assessment answers (each scored 1–5) to a qualitative risk tolerance label. The sum range is 5–25.

| Raw Score Sum | Risk Label              | Typical Profile                              | Notes                                |
|---------------|-------------------------|----------------------------------------------|--------------------------------------|
| 5 – 10        | Conservative            | Near-retirement, capital preservation        | Flag IPS for conservative allocation |
| 11 – 16       | Moderately Conservative | Income-focused, low volatility tolerance     | Benchmark: 30/70 or 40/60            |
| 17 – 20       | Moderate                | Balanced growth and income                   | Most common for mid-career clients   |
| 21 – 23       | Moderately Aggressive   | Growth-oriented, 10+ year horizon            | Benchmark: 70/30 or 80/20            |
| 24 – 25       | Aggressive              | Maximum growth, long horizon, high tolerance | Flag IPS for advisor review          |

## 7. AI Integration

### 7.1 Model

Model: claude-haiku-4-5-20251001 | Max tokens: 4096 | Temperature: default (1.0)

Haiku is selected for cost efficiency (~\$0.003 per run) and sufficient output quality for structured JSON generation. For higher-stakes production deployments, upgrading to claude-sonnet-4-6 would improve IPS narrative quality.

### 7.2 System Prompt Design

The system prompt instructs Claude to respond exclusively in a four-key JSON object. Key design decisions:

- Strict JSON-only output — prevents prose wrapping that would break JSON.parse()
- First-name personalisation directive — ensures narrative reads as client-specific, not templated
- [ADVISOR REVIEW: reason] flag directive — surfaces items needing human oversight without blocking output
- No number invention rule — prevents hallucinated asset values or returns
- IPS ‘real starting point’ instruction — pushes toward substantive output rather than fill-in-the-blank

## 7.3 User Prompt Structure

lib/prompt.ts builds a deterministic plain-text prompt from the collected form data, passing the pre-computed risk label alongside raw answers so Claude can reference both. Goals are emitted in priority order (array index = priority rank).

## 7.4 Error Handling

The API route wraps `JSON.parse()` in a try/catch. On parse failure it returns HTTP 500 with both an error message and the raw string, enabling debugging. The frontend surfaces an inline error with a “Try again” retry trigger.

# 8. Component Reference

| Component        | Path                                | Purpose                                            | Key Props                         |
|------------------|-------------------------------------|----------------------------------------------------|-----------------------------------|
| Home (page)      | app/page.tsx                        | Form state machine, step router, generate() call   | formData, step, result, loading   |
| ProgressBar      | components/ProgressBar.tsx          | Step indicator with completed checkmarks           | steps[], current                  |
| Step1BasicInfo   | components/steps/Step1BasicInfo.tsx | Text inputs and selects for personal info          | data, onChange, onNext            |
| Step2Financial   | components/steps/Step2Financial.tsx | Toggle-button selects for income, assets, accounts | data, onChange, onNext, onBack    |
| Step3Goals       | components/steps/Step3Goals.tsx     | Dynamic goal list with quick-add suggestions       | data, onChange, onNext, onBack    |
| Step4Risk        | components/steps/Step4Risk.tsx      | 5-question grid of option buttons                  | answers, onChange, onNext, onBack |
| Section (inline) | app/page.tsx                        | Output card with title and per-section Copy button | title, content                    |

## 9. API Route Specification

| Field          | Detail                                                                      |
|----------------|-----------------------------------------------------------------------------|
| Method         | POST                                                                        |
| Path           | /api/onboard                                                                |
| Request Body   | JSON-serialised ClientFormData                                              |
| Response (200) | JSON-serialised OnboardingPacket                                            |
| Response (500) | { error: string, raw: string }                                              |
| Auth           | None (API key is server-side only via ANTHROPIC_API_KEY env var)            |
| Timeout        | Vercel default function timeout (10s on free tier; upgrade for reliability) |

## 10. Deployment

### 10.1 Local Development

```
cd ria-onboarding-agent
# Add your API key to .env.local
echo 'ANTHROPIC_API_KEY=sk-ant-...' > .env.local
npm run dev
# Open http://localhost:3000
```

### 10.2 Vercel Deployment

```
npx vercel
# When prompted, add environment variable:
# ANTHROPIC_API_KEY = sk-ant-...
```

The `.env.local` file is covered by the default `.gitignore` (`.env*`) and will never be committed to source control.

### 10.3 Environment Variables

| Variable          | Where                         | Required | Notes                                     |
|-------------------|-------------------------------|----------|-------------------------------------------|
| ANTHROPIC_API_KEY | .env.local / Vercel dashboard | Yes      | Server-side only, never exposed to client |

## 11. Cost Estimate

| Model             | Est. Cost / Run | 1,000 Runs / Month | Notes                                      |
|-------------------|-----------------|--------------------|--------------------------------------------|
| claude-haiku-4-5  | ~\$0.003        | ~\$3.00            | Default — suitable for demo and low volume |
| claude-sonnet-4-6 | ~\$0.02         | ~\$20.00           | Higher quality IPS narratives              |

Cost is driven by output token volume. The IPS section alone is 400–600 words, making this slightly more expensive per run than simpler summarisation tools, but still negligible for demo and pilot use.

## 12. Known Limitations & Production Gaps

| Area               | Current State             | Impact                                             | Recommended Fix                                |
|--------------------|---------------------------|----------------------------------------------------|------------------------------------------------|
| Authentication     | None                      | Anyone with URL can use the tool                   | Add NextAuth or Clerk session-based auth       |
| Data Persistence   | None (in-memory only)     | Output lost on page refresh                        | Add Supabase or PlanetScale for packet storage |
| IPS Compliance     | AI-generated draft only   | Cannot be filed without attorney/compliance review | Add mandatory review workflow flag             |
| Hallucination Risk | Mitigated by prompt rules | Possible on regulatory language                    | Add RAG over firm compliance docs              |
| Vercel Timeout     | 10s on free tier          | Long Claude responses may timeout                  | Upgrade to Pro or add streaming                |
| PDF Export         | Not yet implemented       | Manual copy-paste required                         | Add @react-pdf/renderer ExportButton           |
| Multi-advisor      | Single user               | Cannot serve a firm with multiple advisors         | Add firm accounts and client roster            |

## 13. Future Roadmap

### Phase 2 — Export & Storage

- PDF export with firm branding via @react-pdf/renderer
- Packet storage: save generated packets to Supabase with client ID
- Email delivery: send packet PDF directly to client via Resend



### Phase 3 — CRM Integration

- Redtail CRM push: auto-create client record on packet generation
- Salesforce Financial Services Cloud connector
- Wealthbox integration for boutique RIAs

### Phase 4 — Enterprise

- Multi-advisor firm accounts with role-based access
- Compliance review workflow with approval gates before filing
- Advisor-specific IPS templates and branding
- RAG over firm's existing IPS library for stylistic consistency

## 14. Demo Data

Use the following inputs to demonstrate the tool end-to-end:

| Field          | Demo Value                                                     |
|----------------|----------------------------------------------------------------|
| Name           | Sarah Mitchell                                                 |
| Age            | 47                                                             |
| Marital Status | Married                                                        |
| Dependents     | 2                                                              |
| Employment     | Self-Employed — Owner, boutique interior design firm           |
| Income Range   | \$250,000–\$400,000                                            |
| Assets         | \$1,000,000–\$3,000,000                                        |
| Accounts       | 401(k), Joint brokerage, Savings                               |
| Liabilities    | Mortgage (\$380k remaining), no other debt                     |
| Goal 1         | Retire at 62 — 10–20 years — Highest priority                  |
| Goal 2         | Fund college education (x2) — 2–5 / 5–10 years — High priority |
| Goal 3         | Purchase vacation property — 2–5 years — Medium priority       |
| Goal 4         | Estate / generational wealth — 20+ years — Medium priority     |
| Risk Answers   | [3, 3, 4, 3, 3] → Score 16 → Moderately Conservative           |

## 15. Pre-Submission Checklist

---

- App runs locally (npm run dev)
- All 5 steps navigate forward and backward correctly
- Generation works with sample data (Sarah Mitchell)
- Output renders cleanly for all 4 sections
- Deployed on Vercel with a working shareable link
- .env.local is in .gitignore (confirmed: covered by .env\*)
- README has clear run and deploy instructions
- Loom walkthrough recorded (3–5 min)
- One-page write-up completed
- Submission sent to zach@xylo.ai